Security Code Review Report

Python + Flask User Management REST API

Prepared as an English PDF version of the provided reference material

Scope: Secure code review, vulnerability identification, and remediation guidance

Executive Summary

This review evaluates a fictional Python Flask REST API for user management. The application includes user registration, login, user profile lookup, and an administrative endpoint. Seven security issues were identified, including SQL injection, hardcoded secrets, weak password hashing, authorization flaws, sensitive information exposure, hardcoded privilege checks, and unsafe debug configuration.

Contents

1. Overview
2. Review Methodology and Tools
3. Target Application Code: vulnerable_app.py
4. Vulnerability Findings and Remediation
5. Vulnerability Summary
6. Secure Coding Best Practices
7. Remediated Code Example: secure_app.py
8. Conclusion

1. Overview

This security code review uses Python and Flask as the selected technology stack. The target application is a fictional user management REST API that provides user registration, login, profile lookup, and administrative access. It represents a typical web backend application and includes several common attack surfaces found in real-world services.

The review combines automated static analysis with manual code inspection. This approach helps identify both general vulnerability patterns and logic-level weaknesses that may not be fully captured by automated tools alone.

2. Review Methodology and Tools

2.1 Static Analysis Tools

Tool	Type	Description
Semgrep	SAST	Open-source static analysis tool with support for custom rules. Useful for detecting injection, XSS, hardcoded secrets, and other vulnerability patterns.
Snyk Code	SAST	Developer-friendly SAST tool with IDE and CI/CD integration. Uses AI-assisted analysis to help reduce false positives.
Bearer	SAST	Focuses on data-flow analysis and maps findings to OWASP Top 10 and CWE Top 25 categories.
Bandit	Python-specific SAST	Python security scanner for common issues such as unsafe functions, weak crypto, hardcoded secrets, and insecure configuration.

These tools can be run locally or integrated into a CI/CD pipeline. For this review, Semgrep is treated as the primary automated scanner and is supplemented by manual code walkthrough.

2.2 Review Framework

The review follows the systematic approach recommended by the OWASP Code Review Guide. The analysis focuses on the following security dimensions:

- Input validation - whether all external inputs are validated and sanitized.
- Output encoding - whether data returned to clients is encoded appropriately for its context.
- Authentication and authorization - whether identity verification and access control are complete and correctly enforced.
- Error handling and logging - whether exceptions are handled safely and sensitive information is not leaked.
- Dependency security - whether third-party libraries contain known vulnerabilities.
- Cryptographic practices - whether sensitive data is protected using strong, modern algorithms.
- Session management - whether sessions are protected and CSRF defenses are considered where applicable.

3. Target Application Code: vulnerable_app.py

The following code is the vulnerable Flask application selected for review. It contains the key routes and implementation details relevant to the findings.

```
import sqlite3
import hashlib
import os
from flask import Flask, request, jsonify, session


app = Flask(__name__)
app.secret_key = "hardcoded_secret_key_123"  # Hardcoded secret key


DB_PASSWORD = "admin123"  # Hardcoded database password


def get_db_connection():
    conn = sqlite3.connect('users.db')
    return conn


@app.route('/register', methods=['POST'])
def register():
    username = request.json.get('username')
    password = request.json.get('password')


    # Uses MD5 for password hashing (weak cryptography)
    hashed = hashlib.md5(password.encode()).hexdigest()


    conn = get_db_connection()
    # SQL injection vulnerability: string concatenation
    conn.execute(f"INSERT INTO users (username, password) VALUES ('{username}', '{hashed}')")
    conn.commit()
    conn.close()
    return jsonify({"message": "User registered"})


@app.route('/login', methods=['POST'])
def login():
    username = request.json.get('username')
    password = request.json.get('password')


    conn = get_db_connection()
    # SQL injection vulnerability
    cursor = conn.execute(
        f"SELECT * FROM users WHERE username = '{username}' "
        f"AND password = '{hashlib.md5(password.encode()).hexdigest()}'"
    )
    user = cursor.fetchone()
    conn.close()


    if user:
        session['username'] = username
        return jsonify({"message": "Login successful"})
    return jsonify({"error": "Invalid credentials"}), 401


@app.route('/user/<username>', methods=['GET'])
def get_user(username):
    # Missing authorization check: any logged-in user can view any profile
    if 'username' not in session:
        return jsonify({"error": "Unauthorized"}), 401


    conn = get_db_connection()
    # SQL injection vulnerability
    cursor = conn.execute(f"SELECT * FROM users WHERE username = '{username}'")
    user = cursor.fetchone()
    conn.close()


    if user:
        # Information disclosure: returns password hash
        return jsonify({"username": user[0], "password": user[1]})
    return jsonify({"error": "User not found"}), 404


@app.route('/admin', methods=['GET'])
def admin_panel():
    # Hardcoded administrator check
    if session.get('username') == 'admin':
        return jsonify({"data": "Sensitive admin data"})
```

```
    return jsonify({"error": "Forbidden"}), 403

if __name__ == '__main__':
    app.run(debug=True)  # Debug mode enabled
```

4. Vulnerability Findings and Remediation

Finding 1: SQL Injection

Severity	Critical
Location	/register, /login, and /user/<username> routes
CWE / OWASP	CWE-89 | OWASP A03:2021 - Injection
Recommended Fix	Use parameterized queries or prepared statements instead of string concatenation.

The application inserts user-controlled values directly into SQL queries using f-strings. This allows an attacker to manipulate query logic and may lead to authentication bypass, unauthorized data access, or database tampering.

```
cursor = conn.execute(
    "SELECT * FROM users WHERE username = ? AND password = ?",
    (username, hashed_password)
)
```

Finding 2: Hardcoded Secrets and Credentials

Severity	Critical
Location	app.secret_key and DB_PASSWORD
CWE / OWASP	CWE-259 | OWASP A07:2021 - Identification and Authentication Failures
Recommended Fix	Load secrets from environment variables or a dedicated secrets management service.

The Flask secret key and database password are hardcoded in the source code. If the repository or deployed code is exposed, attackers may gain access to sensitive secrets and potentially forge or tamper with sessions.

```
import os


app.secret_key = os.environ.get("SECRET_KEY")
DB_PASSWORD = os.environ.get("DB_PASSWORD")
```

Finding 3: Weak Password Hashing Algorithm

Severity	High
Location	hashlib.md5(password.encode()).hexdigest()
CWE / OWASP	CWE-327 | OWASP A02:2021 - Cryptographic Failures
Recommended Fix	Use bcrypt, Argon2, or PBKDF2 with a unique salt and an appropriate work factor.

Passwords are stored using MD5. MD5 is fast and unsuitable for password storage because it can be brute-forced at high speed and lacks password-specific protections such as work factors and salts.

```
import bcrypt
```

```
hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
# Verification:
bcrypt.checkpw(password.encode(), stored_hash)
```

Finding 4: Missing Authorization Check / IDOR

Severity	High
Location	/user/<username> route
CWE / OWASP	CWE-639 | OWASP A01:2021 - Broken Access Control
Recommended Fix	Verify resource ownership and allow administrative access only when the user has an appropriate role.

The endpoint checks whether the user is authenticated but does not verify whether the authenticated user is allowed to access the requested profile. This creates an Insecure Direct Object Reference issue.

```
@app.route("/user/<username>", methods=["GET"])
def get_user(username):
    current_user = session.get("username")
    if not current_user:
        return jsonify({"error": "Unauthorized"}), 401


    if current_user != username and session.get("role") != "admin":
        return jsonify({"error": "Forbidden"}), 403
```

Finding 5: Sensitive Information Disclosure

Severity	Medium
Location	/user/<username> response body
CWE / OWASP	CWE-200 | OWASP A01:2021 - Broken Access Control
Recommended Fix	Never return password hashes, tokens, or other secret fields in API responses.

The API returns the user password hash in the response. Even though a hash is not the plaintext password, exposing it gives attackers material for offline password cracking.

```
return jsonify({"username": user["username"], "email": user["email"]})
```

Finding 6: Hardcoded Privilege Check

Severity	Medium
Location	/admin route: session.get('username') == 'admin'
CWE / OWASP	CWE-285 | OWASP A01:2021 - Broken Access Control
Recommended Fix	Implement role-based access control (RBAC) and store the user role or permissions after authentication.

Administrative access is based on a hardcoded username rather than a role or permission model. This is inflexible and prone to bypass or misconfiguration.

```
@app.route("/admin", methods=["GET"])
@roles_required("admin")
def admin_panel():
    return jsonify({"data": "Sensitive admin data"})
```

Finding 7: Debug Mode Enabled

Severity	Medium
Location	app.run(debug=True)
CWE / OWASP	CWE-489 | Not directly mapped
Recommended Fix	Control debug mode using environment-specific configuration and keep it disabled in production.

Running Flask with debug mode enabled can expose stack traces, internal implementation details, and potentially an interactive debugger in unsafe environments.

```
debug_mode = os.environ.get("FLASK_DEBUG", "False").lower() == "true"
app.run(debug=debug_mode)
```

5. Vulnerability Summary

ID	Vulnerability Type	Severity	CWE	OWASP Mapping
V-01	SQL Injection	Critical	CWE-89	A03:2021
V-02	Hardcoded Secrets and Credentials	Critical	CWE-259	A07:2021
V-03	Weak Cryptography / MD5 Password Hashing	High	CWE-327	A02:2021
V-04	Insecure Direct Object Reference / Missing Authorization	High	CWE-639	A01:2021
V-05	Sensitive Information Disclosure	Medium	CWE-200	A01:2021
V-06	Hardcoded Permission Check	Medium	CWE-285	A01:2021
V-07	Debug Mode Enabled	Medium	CWE-489	Not directly mapped

6. Secure Coding Best Practices

6.1 Input Validation and Output Encoding

- Validate all user input on the server side using a whitelist approach wherever possible.
- Use context-aware output encoding before rendering data into HTML or other client-facing contexts.
- Use parameterized queries for all database access.

6.2 Authentication and Authorization

- Use mature authentication libraries or frameworks such as Flask-Login where appropriate.
- Apply the principle of least privilege.
- Use explicit authorization checks for every sensitive resource access.
- Add CSRF protection for session-based state-changing requests.

6.3 Cryptography and Secret Management

- Use bcrypt, Argon2, or PBKDF2 for password storage.

- Store secrets in environment variables or a secrets manager, not in source code.
- Use HTTPS for all sensitive client-server communication.

6.4 Error Handling and Logging

- Use centralized exception handling and avoid exposing stack traces to clients.
- Log security-relevant events such as failed logins and denied access attempts.
- Do not log passwords, tokens, session IDs, or other sensitive data.

6.5 Dependency Management

- Scan dependencies regularly using tools such as safety, pip-audit, or Snyk.
- Update vulnerable dependencies promptly and track remediation status.

6.6 Secure Development Process

- Integrate security checks into the CI/CD pipeline.
- Maintain secure coding standards for the development team.
- Include security criteria in pull request and code review checklists.

7. Remediated Code Example: secure_app.py

The following example demonstrates how the vulnerable application can be improved. It uses environment-based configuration, bcrypt password hashing, parameterized SQL queries, resource ownership checks, and role-based access control.

```
import os
import sqlite3
import bcrypt
from flask import Flask, request, jsonify, session
from functools import wraps


app = Flask(__name__)
app.secret_key = os.environ.get('SECRET_KEY')  # Loaded from environment


def get_db_connection():
    conn = sqlite3.connect('users.db')
    conn.row_factory = sqlite3.Row
    return conn


def roles_required(*roles):
    def decorator(f):
        @wraps(f)
        def decorated(*args, **kwargs):
            if 'role' not in session or session['role'] not in roles:
                return jsonify({"error": "Forbidden"}), 403
            return f(*args, **kwargs)
        return decorated
    return decorator


@app.route('/register', methods=['POST'])
def register():
    username = request.json.get('username')
    password = request.json.get('password')


    # Input validation
    if not username or not password or len(password) < 8:
        return jsonify({"error": "Invalid input"}), 400


    # Strong password hashing
    hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())


    conn = get_db_connection()
    # Parameterized query prevents SQL injection
    conn.execute(
        "INSERT INTO users (username, password, role) VALUES (?, ?, ?)",
        (username, hashed, 'user')
```

```
    )
    conn.commit()
    conn.close()
    return jsonify({"message": "User registered"}), 201


@app.route('/login', methods=['POST'])
def login():
    username = request.json.get('username')
    password = request.json.get('password')


    conn = get_db_connection()
    cursor = conn.execute("SELECT * FROM users WHERE username = ?", (username,))
    user = cursor.fetchone()
    conn.close()


    if user and bcrypt.checkpw(password.encode(), user['password']):
        session['username'] = username
        session['role'] = user['role']
        return jsonify({"message": "Login successful"})
    return jsonify({"error": "Invalid credentials"}), 401


@app.route('/user/<username>', methods=['GET'])
def get_user(username):
    current_user = session.get('username')
    if not current_user:
        return jsonify({"error": "Unauthorized"}), 401


    # Authorization check: users can view themselves; admins can view all
    if current_user != username and session.get('role') != 'admin':
        return jsonify({"error": "Forbidden"}), 403


    conn = get_db_connection()
    cursor = conn.execute(
        "SELECT username, role, email FROM users WHERE username = ?",
        (username,)
    )
    user = cursor.fetchone()
    conn.close()


    if user:
        return jsonify(dict(user))
    return jsonify({"error": "User not found"}), 404


@app.route('/admin', methods=['GET'])
@roles_required('admin')
def admin_panel():
    return jsonify({"data": "Sensitive admin data"})


if __name__ == '__main__':
    debug_mode = os.environ.get('FLASK_DEBUG', 'False').lower() == 'true'
    app.run(debug=debug_mode)
```

8. Conclusion

This security review identified seven vulnerabilities in the target Flask application. The findings cover multiple OWASP Top 10 categories, including injection, cryptographic failures, broken access control, hardcoded secrets, and insecure debug configuration.

The main lessons from this review are:

- Never trust user input. Input validation and parameterized queries are essential baseline controls.
- Never hardcode secrets. Use environment variables or a secrets management platform.
- Never expose unnecessary sensitive data. API responses should include only the minimum required fields.
- Security should be built into the development lifecycle through automated tools, manual review, and secure coding standards.

Treating security review as a routine part of the software development lifecycle, rather than as a last-minute activity, significantly reduces the chance of introducing preventable vulnerabilities into production environments.